Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3597503.3639187

RESEARCH-ARTICLE

Using an LLM to Help With Code Understanding

DAYE NAM, Carnegie Mellon University, Pittsburgh, PA, United States

ANDREW MACVEAN, Google LLC, Mountain View, CA, United States

VINCENT HELLENDOORN, Carnegie Mellon University, Pittsburgh, PA, United States

BOGDAN VASILESCU, Carnegie Mellon University, Pittsburgh, PA, United States

BRAD MYERS, Carnegie Mellon University, Pittsburgh, PA, United States

Open Access Support provided by:

Carnegie Mellon University

Google LLC

PDF Download
3597503.3639187.pdf
07 April 2026
Total Citations: 253
Total Downloads: 21508

Published: 12 April 2024

Citation in BibTeX format

ICSE '24: IEEE/ACM 46th International Conference on Software Engineering
April 14 - 20, 2024
Lisbon, Portugal

Conference Sponsors:
SIGSOFT

ICSE '24: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (May 2024)
https://doi.org/10.1145/3597503.3639187
ISBN: 9798400702174

Using an LLM to Help With Code Understanding

Daye Nam
Carnegie Mellon University
U.S.A.
dayen@cs.cmu.edu

Andrew Macvean
Google, Inc.
U.S.A.
amacvean@google.com

Vincent Hellendoorn
Carnegie Mellon University
U.S.A.
vhellend@andrew.cmu.edu

Bogdan Vasilescu
Carnegie Mellon University
U.S.A.
vasilescu@cmu.edu

Brad Myers
Carnegie Mellon University
U.S.A.
bam@cs.cmu.edu

ABSTRACT

Understanding code is challenging, especially when working in new and complex development environments. Code comments and documentation can help, but are typically scarce or hard to navigate. Large language models (LLMs) are revolutionizing the process of writing code. Can they do the same for helping understand it? In this study, we provide a first investigation of an LLM-based conversational UI built directly in the IDE that is geared towards code understanding. Our IDE plugin queries OpenAI's GPT-3.5-turbo model with four high-level requests *without* the user having to write explicit prompts: to explain a highlighted section of code, provide details of API calls used in the code, explain key domain-specific terms, and provide usage examples for an API. The plugin also allows for open-ended prompts, which are automatically contextualized to the LLM with the program being edited. We evaluate this system in a user study with 32 participants, which confirms that using our plugin can aid task completion more than web search. We additionally provide a thorough analysis of the ways developers use, and perceive the usefulness of, our system, among others finding that the usage and benefits differ between students and professionals. We conclude that in-IDE prompt-less interaction with LLMs is a promising future direction for tool builders.

ACM Reference Format:
Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to Help With Code Understanding. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE '24), April 14–20, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3597503.3639187

1 INTRODUCTION

Building and maintaining software systems requires a deep understanding of a codebase. Consequently, developers spend a significant amount of time searching and foraging for the information they need and organizing and digesting the information they find [30, 31, 34, 44, 48, 57]. Understanding code, however, is a challenging task; developers need to assimilate a large amount of information about the semantics of the code, the intricacies of the APIs used, and the relevant domain-specific concepts. Such information is often scattered across multiple sources, making it challenging for developers, especially novices or those working with unfamiliar APIs, to locate what they need. Furthermore, much of the relevant information is inadequately documented or spread across different formats and mediums, where it often becomes outdated.

With the growing popularity of large language model (LLM) based code generation tools [26, 54, 67], the need for information support for code understanding is arguably growing even higher. These tools can generate code automatically, even for developers with limited coding skills or domain knowledge. This convenience comes at a cost, however – developers may receive code they don't understand [24, 79]. Indeed, early research on LLM code generation tools has found that developers have a harder time debugging code generated by the LLM and easily get frustrated [40, 71].

Fortunately, LLMs also provide an opportunity in this space, namely by offering on-demand *generation-based information support* for developers faced with unfamiliar code. Compared to general web search queries [74], LLM prompts can allow developers to provide more context, which can enable them to receive information that more precisely aligns with their specific needs, potentially reducing the time spent on sifting through the information obtained from the web to suit their particular requirements. Developers have indeed taken to web-hosted conversational LLM tools, such as ChatGPT, for programming support en masse, but this setup requires them to both context switch and copy the relevant context from their IDEs into the chat system for support.

To explore the potential for generation-based information support directly in the developer's programming environment, we developed a prototype in-IDE LLM information support tool, GILT (Generation-based Information-support with LLM Technology). GILT is capable of generating on-demand information while considering the user's local code context, which we incorporate into the prompts provided to the LLM behind the scenes. This way, we also introduce a novel interaction method with the LLM, *prompt-less interaction*. This option aims to alleviate the cognitive load associated with writing prompts, particularly for developers who possess limited domain or programming knowledge.

As there is still little knowledge about how to best use an LLM for information support (as opposed to just code generation), we evaluate the effectiveness of our prototype tool in an exploratory

This work is licensed under a Creative Commons Attribution International 4.0 License.
ICSE '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0217-4/24/04.
https://doi.org/10.1145/3597503.3639187

user study with 32 participants tasked with comprehending and extending unfamiliar code that involves new domain concepts and Python APIs for data visualization and 3D rendering – a challenging task. Our study quantitatively compares task completion rates and measures of code understanding between two conditions – using the LLM-backed assistant in-IDE versus directly searching the web in a browser – and qualitatively investigates how participants used the tools and their overall satisfaction with this new interaction mode. Concretely, we answer three research questions:

- RQ1: To what extent does GILT affect developers' understanding, task completion time, and task completion rates when faced with unfamiliar code?
- RQ2: How do developers interact with GILT, and to what extent does that differ between the participants?
- RQ3: How do developers perceive the usefulness of GILT?

Our results confirm that there are statistically significant gains in task completion rate when using GILT, compared to a web search, showing the utility of generation-based information support. However, we did not find the utility gains in terms of time and understanding level, leaving room for further improvement. We also discovered that the degree of the benefit varies between students and professionals, and investigated potential reasons behind this.

2 RELATED WORK

2.1 Studies on Developers Information Seeking

In every phase of modern software engineering, developers need to work with unfamiliar code and domains, and how well they learn such code influences their productivity significantly. Therefore, researchers have studied to understand how developers learn and comprehend unfamiliar code and domains [30, 38, 51, 62], especially how they search for and acquire information, as developers need a variety of kinds of knowledge [43, 66, 68, 77]. Particularly, lots of research was done on the information seeking strategies of developers, mostly in general software maintenance [18, 30, 35] or web search settings [7, 59]. Researchers have also studied challenges developers face [14, 23, 32, 58, 72, 74], including difficulties in effective search-query writing and information foraging, and built tools to support developers overcome such challenges [3, 42, 52]. In this work, we explore a way of supporting developers' information needs with generation-based information support using LLMs.

Other efforts were made to understand developers' information seeking within software documentation, which is the main source of information for developers when they learn to use new APIs or libraries. Researchers cataloged problems developers face when using documentation [9, 60, 61] and identified types of knowledge developers report to be critical [46, 60, 61, 70], which were taken into account when we designed GILT for our study.

2.2 Studies on LLM-based Developer Tools

The potential and applicability of LLM-based AI programming tools have been actively studied by many researchers. Numerous empirical studies [17, 22, 37, 65] evaluated the quality of code or explanations generated by LLMs, to test the feasibility of applying LLM into development tools [41, 69, 79] and to Computer Science education [1, 22, 65]. Several studies have also compared LLM-generated code and explanations with those authored by humans without LLM assistance [22, 37, 56], demonstrating that LLMs can offer reasonably good help for developers or students with caution.

Fewer studies have specifically explored the *usefulness* of LLM-based programming tools [2, 29, 40, 49, 63, 64, 71, 75, 79, 79] with actual users or their usage data, and many of these studies have focused on code generation tools like CoPilot [26]. For instance, Ziegler et al.[79] analyzed telemetry data and survey responses to understand developers' perceived productivity with GitHub Copilot, revealing that over one-fifth of suggestions were accepted by actual developers. Several human studies were also conducted. Vaithilingam *et al.* [71] compared the user experience of GitHub Copilot to traditional autocomplete in a user study and found that participants more frequently failed to complete tasks with Copilot, although there was no significant effect on task completion time. Barke [2] investicated further with a grounded theory analysis to understand *how* programmers interact with code-generating models, using Github Copilot as an example. They identified two primary modes of interaction, acceleration or exploration, where Copilot is used to speed up code authoring in small logical units or as a planning assistant to suggest structure or API calls.

Although these studies have increased our understanding of the usefulness and usability of AI programming assistants in general, and some of the insights apply to information support, they do not show the opportunities and challenges of LLM-based tools as information support tools, with a few following exceptions [45, 63]. MacNeil et al. [45] examined the advantages of integrating code explanations generated by LLMs into an interactive e-book focused on web software development, with a user study with sophomores. They found students tend to find LLM-generated explanations to be useful, which is promising, but the study was focused on providing one-directional support in an introductory e-book which is different from user-oriented need-based information support. The Programmer's assistant [63] is the closest to our work. The authors integrated a conversational programming assistant into an IDE to explore other types of assistance beyond code completion. They collected quantitative and qualitative feedback from a human study with 42 participants from diverse backgrounds and found that the *perceived* utility of the conversational programming assistance was high. In our work, we focus on the utility of LLM-based tools to satisfy information needs for *code understanding*, and take a step forward to test the actual utility of an LLM-integrated programming tool by assessing performance measures such as completion rates, time, and participants' code understanding levels.

3 THE GILT PROTOTYPE TOOL

We iteratively designed GILT to explore different modes of interaction with an LLM for information support. GILT is a plugin for the VS Code IDE (Figure 1) that considers user context (the code selected by the user) when querying an LLM for several information support applications.

3.1 Interacting with GILT

There are two ways to interact with the plugin. First, users can select parts of their code (Figure 1-②) and trigger the tool by clicking on "AI Explanation" on the bottom bar (Figure 1-①), or using

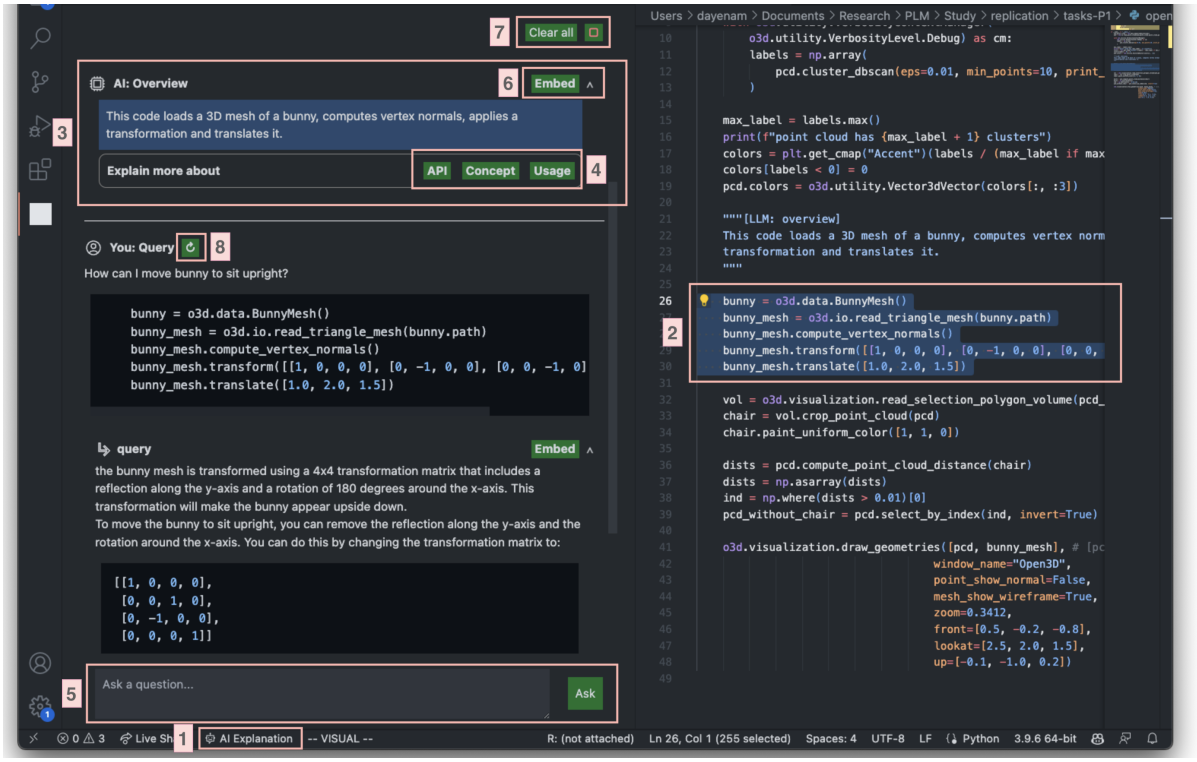


Figure 1: Overview of our prototype. (1) A trigger button; (2) code used as context when prompting LLM; (3) code summary (no-prompt trigger); (4) buttons for further details; (5) an input box for user prompts; (6) options to embed information to code (Embed) and a hide/view button; (7) options to clear the panel (Clear all) and an abort LLM button; (8) a refresh button.

"alt/option + a" as a shortcut, to receive a summary description of the highlighted code (`Overview`). They can then explore further by clicking on buttons (Figure 1-④) for API (`API`), domain-specific concepts (`Concept`), and usage examples (`Usage`), which provide more detailed explanations with preset prompts. The `API` button offers detailed explanations about the API calls used in the code, the `Concept` button provides domain-specific concepts that might be needed to understand the highlighted code fully, and the `Usage` button offers a code example involving API calls used in the highlighted code.

Users can also ask a specific question directly to the LLM via the input box (Figure 1-⑤). If no code is selected, the entire source code is used as context (`Prompt`); alternatively, the relevant code highlighted by the user is used (`Prompt-context`). The model will then answer the question with that code as context. GILT also allows users to probe the LLM by supporting conversational interaction (`Prompt-followup`). When previous LLM-generated responses exist, if a user does not highlight any lines from the code, the LLM generates a response with the previous conversion as context. Users can also reset the context by triggering the tool with code highlighted, or with the Clear all button.

3.2 Our Design Process and Decisions

Focus on understanding. We intentionally did not integrate a code generation feature in the prototype as we wanted to focus on how developers *understand* code.

In-IDE extension. Besides anticipating a better user experience, we designed the prototype as an in-IDE extension to more easily provide the code context to the LLM – participants could select code to use as part of the context for a query.

Pre-generated prompts. We designed buttons that query the LLM with pre-generated prompts (*prompt-less* interaction) to ask about an API, conceptual explanations, or usage examples, as shown in Figure 1-④. We chose these based on API learning theory [14, 33, 46, 68], expecting this may particularly assist novice programmers or those unfamiliar with the APIs/domains or the LLM, as writing efficient search queries or prompts can be difficult for novices [13, 15, 32]. At the same time, we also expected that this could reduce the cognitive burden of users in general in formulating prompts.

For `Overview` and the buttons `API`, `Concept`, `Usage`, we came up with prompt templates after a few iterations. To more efficiently provide the context to LLM, we used the library names and the list of API methods included in the selected code, such as "Please provide a [library name] code example, mainly showing the usage of the following API calls: [list of API methods]" for `Usage`.

Unrestricted textual queries. Users can also directly prompt the LLM (Figure 1-⑤), in which case GILT will automatically add any selected code as context for the query. Internally, the tool adds the selected code as part of the user prompt using pre-defined templates, and requests the LLM to respond based on the code context.

Need-based explanation generation. The tool is pull-based, i.e., it generates an explanation only when a user requests it. Similar to many previous developer information support tools, we wanted to reduce information overload and distraction. We expect that if and when enough context can be extracted from the IDE, hybrid (pull + push) tools will be possible, but this would require more research.

Iterative design updates. We ran design pilot studies and updated our prototype accordingly. For example, we made the code summary as the default action for the tool trigger with code selection, after seeing pilot participants struggling to find parts of code to work on due to their unfamiliarity with libraries and domains. We updated the prompt-based interaction with LLM to support a conversational interface, based on the pilot participants' feedback that they wanted to probe the model based on their previous queries to clarify their intent or ask for further details. Finally, we opted to use GPT-3.5-turbo instead of GPT-4 as planned, after discovering that the response time was too slow in the pilot studies.

4 HUMAN STUDY DESIGN

Participants. We advertised our IRB-approved study widely within the university community (through Slack channels, posted flyers, and personal contacts) and to the public (through Twitter, email, and other channels). We asked each participant about their programming experience and screened out those who reported having a "not at all" experience. We did not ask about their professional programming experience, as the target users of our information support tools are not limited to professional developers. To minimize the possibility of participants knowing solutions, we specifically sought out participants who had not used the libraries included in our study. We accepted participants into the study on a rolling basis to capture a range of programming experience and existing domain knowledge. We compensated each participant with a $25 Amazon Gift card. We recruited 33 participants and conducted 33 studies in total. However, we had to exclude data from one participant from the analysis because they did not follow the instructions for using the extension. In the end, we had 9 women and 23 men participants. Among them, 16 participants identified themselves primarily as students, 1 as software engineer, 2 as data scientists, and 13 as researchers. In the analysis, we divided the participants into two groups (students vs. professionals) based on this. 24 participants had experience with ChatGPT, 15 with Copilot, 5 with Bard, while 7 participants reported no prior use of any LLM-based developer tools. In terms of familiarity with such tools, 14 participants stated that they have either used AI developer tools for their work or always use them for work.

Tasks. The tasks were designed to simulate a scenario in which developers with specific requirements search the web or use existing LLMs to generate code and find similar code that does not precisely match their needs. For each task, we provided participants with a high-level goal of the code, start and goal outputs, and a starter code file loaded into the IDE. In this way, participants had to understand the starter code we provided and make changes to it so that the modified code met the goal requirements. Each task consisted of 4 sub-tasks, to help reduce participant's overhead in planning, as well as to measure the task completion ratio in the analysis. There were some subtle dependencies between the sub-tasks, so we advised participants to follow the order we gave, but they were free to skip. Completing each sub-task required a small change, ranging from a single parameter value update to an addition of a line. The difficulty levels of the sub-tasks varied, but we intentionally designed the first sub-task to be easy so that participants can onboard easily. Sub-tasks also came with start and goal outputs and descriptions (see Figure 2). We did not include tasks that required strong programming knowledge, because our goal was to assess how well participants could understand the code. For the same reason, we provided participants with starter code that was runnable and bug-free.

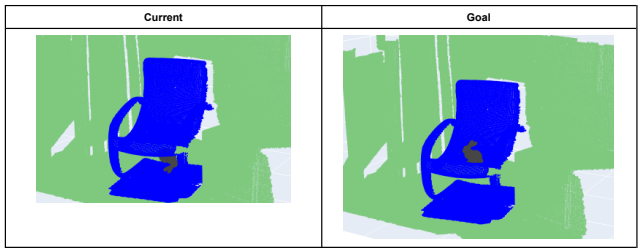


Figure 2: A 3D-rendering example sub-task (open3d-3). With these start and goal outputs, we asked the participants to "Make the bunny sit upright on the chair." See Figure 1 for the corresponding starter code and the tool output.

Our tasks cover both a common and less common domain that a Python developer might encounter in the wild. We chose two domains: data visualization and 3D rendering. These two tasks also allowed participants to easily check their progress, as they produce visual outputs that are comparable with the given goal. For the data visualization task, we used the Bokeh [5] library and asked participants to edit code that visualizes income and expenses data in polar plots. Understanding this code required knowledge of concepts related to visualizing data formats, marks, and data mapping. In the 3D rendering task, we used the Open3d [53] library. This task required knowledge of geometry and computer graphics. Participants were asked to edit code that involved point cloud rendering, transformation, and plane segmentation.

When selecting libraries, we intentionally did not choose the most common ones in their respective domains, to reduce the risk of participants knowing them well. Choosing less common libraries also helped reduce the risk of an outsized advantage of our LLM-powered information generation tool. Responses for popular libraries can be significantly better than those for less commonly used ones, as the quality of LLM-generated answers depends on whether the LLM has seen enough relevant data during training.

The Bokeh starter code consisted of 101 LOC with 11 (6 unique) Bokeh API calls, and the starter code for the Open3D task consisted of 43 LOC with 18 (18 unique) Open3D API calls. The tasks were designed based on tutorial examples in each API's documentation. In the starter codes, we did not include any comments in the code to isolate the effects of the information provided by our prototype or collected from search engines. All necessary API calls were in the starter code so participants did not need to find or add new ones.

In the task descriptions, we tried to avoid using domain-specific or library-specific keywords that could potentially provide participants with direct answers from either search engines or GILT. For instance, we opted to use "make the bunny...", instead of "transform the bunny..." which may have steered participants towards the function `transform` without much thought.

The full task descriptions, starter code, solution for the demo, and the actual tasks are available in our replication package.

Experimental Design. We chose a within-subjects design, with participants using both GILT (treatment) and a search engine (control) for code understanding, but they did so on different tasks. This allowed us to ask participants to rate both conditions and provide comparative feedback about both treatments.

The control-condition participants were not allowed to use our prototype, but they were free to use any search engine to find the information they needed. The treatment-condition participants were encouraged to primarily use our prototype for information support. However, if they could not find a specific piece of information using our prototype, we allowed them to use search engines to find it. This was to prevent participants from being entirely blocked by the LLM. We expected that this was a realistic use case of any LLM-based tool, but it rarely happened during the study. Only 2 participants ended up using search engines during the treatment condition, but they could not complete the tasks even with the search engines.

We counterbalanced the tasks and the order they were presented to participants to prevent carryover effects, resulting in four groups (2 conditions x 2 orders). We used random block assignments when assigning participants to each group. Participants were assigned to each group to balance the self-reported programming and domain experience (data visualization and 3D rendering). For every new participant, we randomly assigned them to a group that no previous participant with the same experience level had been assigned. If all groups had previous participants with the same experience level, we randomly assigned the participant to any of them.

Study Protocol. We conducted the study via a video conferencing tool and in person, with each session taking about 90 minutes; in-person participants also used the video conferencing tool, for consistency. At the beginning of the study, we asked participants to complete a pre-study survey, collecting their demographic information, background knowledge, and experience with LLMs. We also estimated their general information processing and learning styles using a cognitive style survey [20] categorizing participants into two groups per dimension: comprehensive / selective information processing and process-oriented learning / tinkering. The participants were then asked to join our web-based VS Code IDE hosted on GitHub CodeSpaces [25], which provided a realistic IDE inside a web browser with edit, run, test, and debug capabilities, without requiring participants to install any software locally [12]. We then showed them a demo task description and explained what they would be working on during the real tasks. Before their first task in the treatment condition, we provided a tutorial for our plugin using the demo task, introducing each feature and giving three example prompts for the LLM. For the control condition, we did not provide any demo, as we expected every participant to be able to use search engines fluently. For each task, we gave participants 20 minutes to complete as many sub-tasks as they could. During the task, we did not use the think-aloud protocol because we wanted to collect timing data. Instead, we collected qualitative data in the post-survey with open-ended questions. We also collected extensive event and interaction logs during the task. After each task, we asked participants to complete a post-task survey to measure their understanding of the provided code and the API calls therein. At the very end, we asked them to complete a post-study survey where we asked them to evaluate the perceived usefulness and perceived ease of use of each code understanding approach and each feature in GILT. We based our questionnaire on the Technology Acceptance Model (TAM) [36], NASA Task Load Index (TLX) [21], and pre- and post-study questionnaires that were previously used in similar studies [63, 68]. See our replication package for the instruments.

We conducted 33 studies in total, with 33 participants. The initial 18 studies were conducted on a one-on-one basis, while some studies in the latter half (involving 15 participants) were carried out in group sessions, with two to five participants simultaneously and one author serving as the moderator. We took great care to ensure that participants did not interrupt each other or share their progress. As mentioned before, we excluded one participant's data and used 32 participants' for the analysis. We discovered this issue after the study, as this participant was part of the largest group session (with five participants).

5 RQ1: EFFECTS OF GILT

In this section, we report on the effectiveness of using GILT in understanding unfamiliar code.

5.1 Data Collection

Code understanding. To evaluate the effectiveness of each condition, we used three measurements: (1) Task completion time: to complete each sub-task; (2) Task progress: we rated the correctness of the participants' solution to each sub-task and measured how many sub-tasks they correctly implemented; and (3) Understanding level: we cross-checked participants' general understanding of the starter code by giving them sets of quiz questions about the APIs included in the starter code. Each set contained three questions, requiring an in-depth understanding of the functionalities of each API call and the application domains. To measure the effect of using GILT and search engines, we excluded the sub-tasks data if participants *guessed* the solution without using the tool (i.e., zero interaction with the tool) or search engines before completing it (i.e., no search queries).

Prior knowledge. To control for prior knowledge, we used self-reported measures of participants' programming and domain experience. We expected more programming experience, especially in the specific domain, to lead to a faster understanding of code.

Experience in AI developer tools. Crafting effective prompts for LLM-based tools requires trial and error, even for NLP experts [13, 16, 76]. Therefore, we asked participants about their experience with LLM-based developer tools. We expected participants' familiarity with other AI tools to affect their usage of the LLM-based information support tool, especially the use of free-form queries, and lead to more effective use of the extension than participants without such experience.

Table 1: Summaries of regressions estimating the effect of using the prototype. Each column summarizes the model for a different outcome variable. We report the coefficient estimates with the standard errors in parentheses.

	Progress (1)	*Time(s)* (2)	*Underst.* (3)	Progress *Pros*	Progress *Students*
Constant	0.41	312.65	−1.81**	−0.38	1.82**
	(0.49)	(185.33)	(0.89)	(0.68)	(0.83)
Domain experience	0.13*	23.14	0.41***	0.16	0.04
	(0.07)	(25.40)	(0.12)	(0.09)	(0.11)
Program. experience	−0.10	−23.67	0.20	0.01	−0.37*
	(0.12)	(43.53)	(0.22)	(0.17)	(0.21)
AI tool familiarity	−0.01	7.70	−0.09	0.07	−0.10
	(0.07)	(27.04)	(0.14)	(0.11)	(0.10)
Uses GILT	0.47***	−9.10	0.29	0.57**	0.29
	(0.16)	(57.26)	(0.28)	(0.22)	(0.25)
R^2	0.173	0.022	0.202	0.341	0.137
Adj. R^2	0.117	−0.046	0.148	0.243	0.010

Note: *p <0.1; **p <0.05; ***p <0.01.

5.2 Methodology

To answer RQ1, we compared the effectiveness of using a GILT with traditional search engines for completing programming tasks by estimating regression models for three outcome variables. For task progress and code understanding, we used quasi-Poisson models because we are modeling count variables, and for the task completion time, we used a linear regression model.

To account for potential confounding factors, we included task experience, programming experience, and LLM knowledge as control variables in our models. Finally, we used a dummy variable (`uses_GILT`) to indicate the condition (using GILT vs. using search engines). We considered mixed-effects regression but used fixed effects only, since each participant and task appear only once in the two conditions (with and without GILT). For example, for the task completion time response, we estimate the model:

```
completion_time ~ domain_experience + programming_experience
                  + AI_tool_familiarity + uses_GILT
```

The estimated coefficient for the `uses_GILT` variable indicates the effect of using GILT while holding fixed the effects of programming experience, domain experience, and LLM knowledge.

5.3 Results

Table 1 columns (1)-(3) display the regression results for three response variables. The task progress model (Table 1-(1)) shows a significant difference between the two conditions, with participants in the GILT condition completing statistically significantly more sub-tasks (0.47 more, $p < 0.01$) than those who used search engines, controlling for experience levels and AI tool familiarity. This indicates that GILT may assist users in making more progress in their tasks compared to search engines.

On the other hand, models (2) and (3) fail to show any significant difference in completion time and code understanding quiz scores between conditions. This suggests that users in the GILT condition do not complete their tasks at a sufficiently different speed or have a sufficiently different level of understanding than those in the control group, given the statistical power of our experiment.

In summary, the results suggest that GILT may help users make more progress in their tasks without changing, for better or worse, their speed and code understanding abilities.

5.4 Additional Analysis

After observing the significant effect of GILT on task progress, we dove deeper to examine whether all participants benefited equally from the tool. To do this, we divided the participants into two distinct groups based on their self-reported occupations (professionals and students) and estimated the effects of GILT usage in each group.[1] We opted for these groups as we did not have any prior theoretical framework to guide our grouping choices, and it provided a simple yet effective approach to group participants with multiple dimensions, including programming experience, skills, and attitude toward programming.

Although both groups were more successful when using the tool, there were notable differences in their performance gains. To better understand these variations, we estimated coefficients for each group (Table 1-Pros and -Students) and observed that the impact of GILT was significant only in the Pros group model. Specifically, professionals completed 0.57 more sub-tasks with GILT support compared to when they used search engines, whereas students did not experience significant gains. These findings suggest that the degree of benefit provided by GILT may vary depending on participants' backgrounds or skills.

Summary RQ1

There are statistically significant gains in task completion rate when using GILT, compared to a web search, but the degree of the benefit varies between students and professionals.

6 RQ2: GILT USAGE

In this section, we focus on how participants interacted with GILT, their perception of the importance of different features, and how different factors correlate with the feature usage.

6.1 Usage of Features

To analyze in more detail how participants actually used the tool, we instrumented GILT and recorded participants' event and interaction logs. The logs allowed us to count the number of times participants triggered each feature, and in what order. To supplement the usage data, participants were asked to rate the importance of each feature in a post-task survey. We used these ratings to triangulate our findings from the usage data.

Figure 3 summarizes the sequences of GILT features used by participants in the treatment condition. On average, to complete their tasks in this condition, participants interacted with the LLM via GILT 15.34 times. The number of interactions per participant ranged

[1]We considered but decided against, modeling interaction effects as they would have required more statistical power.

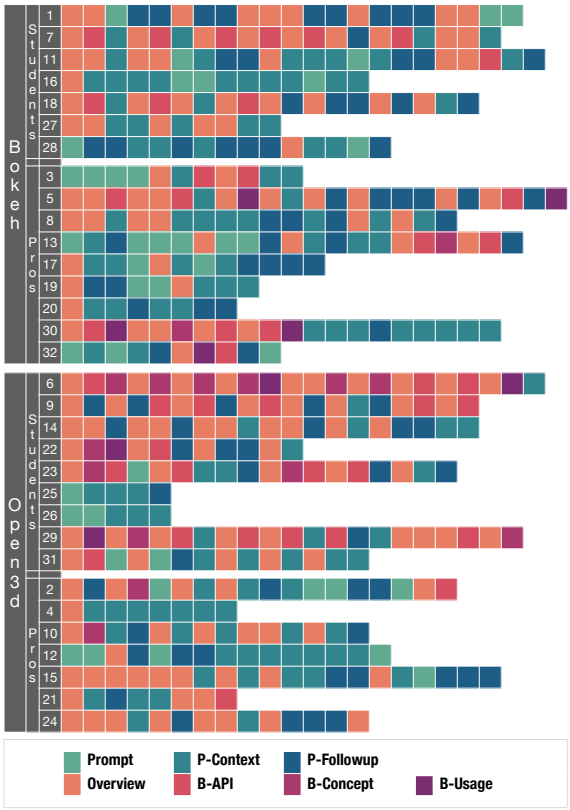


Figure 3: The sequences of feature usage in GILT. Each row corresponds to an individual participant, and the color cells are arranged chronologically, from left to right.

from a minimum of 5 to a maximum of 23. The `Overview` feature was the most frequently used method to interact with the LLM, with an average of 4.76 activations per participant. Many participants also used `Overview` as their first feature, possibly because it requires minimal effort, with just a single click, in contrast to other features that necessitated the formulation of queries by participants, and perhaps also because some of the buttons (e.g., `Concept`) required first using the `Overview` feature. Participants also frequently used `Prompt-context` (4.12 times) and `Prompt-followup` (2.88 times). General prompting without code context was used less frequently (1.27 times). While participants generally used buttons less frequently, some used them more frequently than queries (e.g., P29), indicating personal preferences in prompt-based and prompt-less interactions. Specifically, the `API` button was used 1.24 times, the `Concept` button 0.45 times, and the `Usage` button 0.24 times on average.

The reported importance of the features by participants (see Figure 4) generally corresponds to the observed usage data. Most of the participants (97%) responded that the ability to directly ask questions to the LLM was extremely/very important, whereas their reported usefulness of the buttons varied. The reported importance of the overview feature (53% extremely/very important) was relatively low compared to the actual use, suggesting that participants may not have used the summary description provided by the overview but instead used it as context for further prompting or to activate buttons.

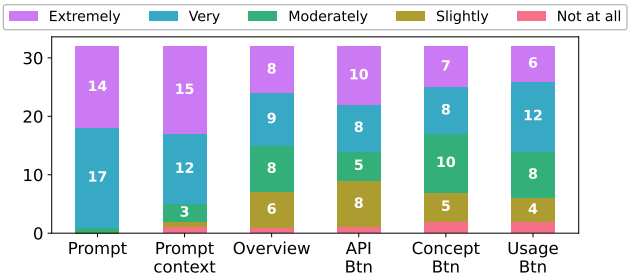


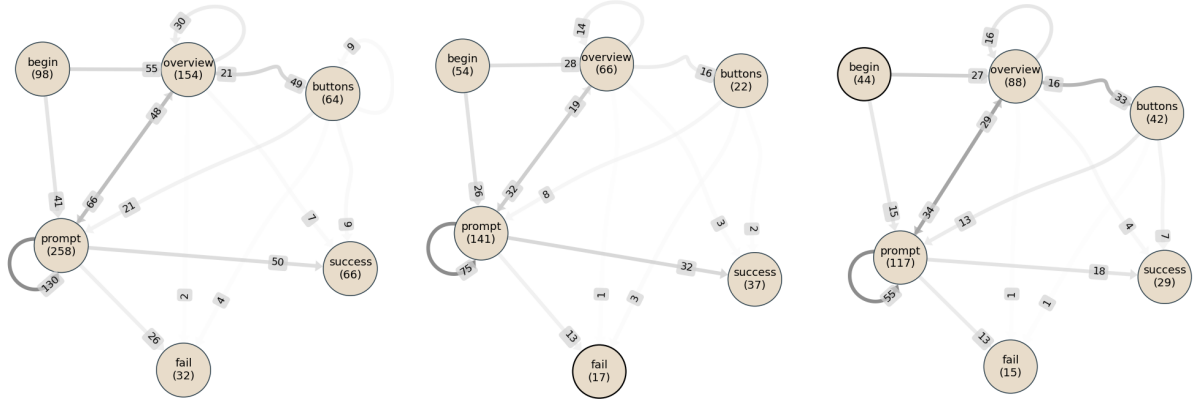
Figure 4: Participants' report on the importance of GILT features.

To further investigate participants' interaction with the tool, we created transition graphs (Figure 5) using sequences of feature use events for each sub-task, using both the sub-tasks successfully completed by participants and those that resulted in failure (due to incorrect answers or timeouts). Out of the potential total of 128 sub-tasks (32 participants × 4 sub-tasks), 98 sub-tasks were started before the time ran out. In understanding the transition graph, we focused on the last feature in each participant's sequence, with an assumption that when a participant completes a task, it is likely that the information from the last interaction satisfied their information needs. Among the sub-tasks they successfully completed, a substantial majority (75%) originated from prompt-based interactions. At the same time, 83% of the failed tasks were also preceded by prompt-based interactions, so prompt-based interactions were not particularly likely to result in successful information seeking.

6.2 Professionals vs. Students

To better understand the experiences of professionals and students (see Section 5.4), we compared the transition graphs for both groups (Figure 5 (b) and (c)). Notable distinctions emerged in terms of the features more likely influencing the success and failure of sub-tasks. Specifically, for professionals, a majority (86%) of successful sub-tasks originated from `prompt`, whereas for students, this percentage (62%) was statistically significantly lower ($\chi^2(1, 66)$, $p < .05$). The success rate of prompt-based interaction was also higher among professionals (71%: 32 out of 45) compared to students (58%: 18 out of 31). Conversely, the success rate of the overview and buttons for professionals (56%: 5 out of 9) was lower than that of students (85%: 11 out of 13). These results may indicate that students, possibly with less experience in information seeking for programming, encounter challenges in formulating effective prompts compared to the professionals, and rely more on prompt-less interaction. However, we can also infer that prompt-less interaction is still not sufficient to compete with the benefits of prompt-based interaction with the current design, as they only accounted for less than 40% of the completed tasks.

To further investigate the differences in the two groups' prompt engineering, we analyzed the text of the prompts they wrote, by comparing the frequencies of bi-, tri-, and quadrigrams in the prompts. Table 2 presents the list of n-grams that showed divergent usage between the two groups. One notable observation is that the n-grams used by the professional group include more effective keywords, or they revise the prompts to incorporate such keywords.



(a) All (b) Professionals (c) Students

Figure 5: Transition Graphs for User Interaction. Each node displays the number of times users interacted with respective features, and each edge indicates the counted number of transitions between the connected features. For space and readability reasons, `Prompt`, `Prompt-Context`, and `Prompt-Followup` are merged into `prompt`, and `API`, `Concept`, and `Usage` are merged into `buttons`. Counts lower than 5 are omitted except for the edges connected to the 'Success' and 'Fail' nodes.

Table 2: Frequencies of n-grams used differently in prompts by professionals and students. For clarity, we only include n-grams used uniquely by one of the two groups, with a frequency difference of more than 2. If multiple n-grams share the same longer n-gram, we report only the superset.

Sub-task	n-gram	Pro.	Stu.
bokeh-2	('align', 'text')	3	0
	('flip', 'label')	0	3
bokeh-3	('annular', 'wedge')	6	0
	('grid', 'annular', 'wedge')	3	0
	('first', 'pie')	3	0
	('pie', 'chart')	3	0
	('add', 'legend')	0	4
	('tell', 'line', 'need', 'change')	0	3
o3d-3	('sit', 'upright', 'chair')	4	0
	('make', 'bunny', 'sit', 'upright', 'chair')	3	0

For instance, in the bokeh-3 sub-task, none of the participants in the student group used the critical keyword "annular wedge," which is essential for generating the information needed to solve the task, although it was used multiple times in the provided starting code. Instead, students tended to use more general keywords or keywords that had a different concept in the library (e.g., "legend") and faced difficulties in effectively revising the prompts. In addition, more participants in the professional group demonstrated proficiency in refining their prompts by providing further specifications. For example, one participant revised the prompt from "How to change the position of the bunny to 180 degrees" to "How to transform the bunny_mesh to 180 degrees." We infer that the difference in the benefit received from GILT by the two groups can be at least partially attributed to their proficiency in prompt engineering.

6.3 Other Factors Associated with Feature Use

During the pilot studies, we observed that participants approached the tasks differently depending on their familiarity with other LLM-based tools, and styles of information processing and learning as observed in many previous studies on software documentation and debugging [4, 46, 47]. Thus, we tested whether the GILT feature use correlates with factors other than their experience.

Hypotheses. Out of two information processing styles [8, 11, 19], people who exhibit a "selective information processor" tendency focus on the first promising option, pursuing it deeply before seeking additional information. On the other hand, people who are "comprehensive information processors" tend to gather information broadly to form a complete understanding of the problem before attempting to solve it. Based on these processing styles, we hypothesized that selective processors would utilize GILT's `Prompt-followup`, as they would prefer to use a depth-first strategy.

In terms of learning styles [8, 55], "process-oriented learners" prefer tutorials and how-to videos, while "tinkerers" like to experiment with features to develop their own understanding of the software's inner workings. Consequently, we hypothesized that tinkerers would use GILT less often, as they would prefer to tinker with the source code rather than collect information from the tool.

We also expected that participants who were already familiar with LLM-based tools would use prompt-based interaction in general (`Prompt`), especially the chat feature, more frequently, as they would already be accustomed to using chat interfaces to interact with LLMs. Conversely, we posited that participants with less experience with such tools would use the buttons more, as prompt engineering might be less familiar to them and place greater cognitive demands on them.

Methodology. To test for associations between GILT features used and the factors above we again used multiple regression analysis. We estimated three models, each focused on one particular feature. For each model, the dependent variable was the feature usage count,

Table 3: Summaries of regressions testing for associations between the user factors and the feature usage counts. Each column summarizes a regression modeling a different outcome variables. We report the coefficient estimates with their standard errors in parentheses.

	Prompt (1)	Followup (2)	All (3)
Constant	1.39***	−0.82	2.43***
	(0.31)	(0.69)	(0.27)
AI tool familiarity	0.19**	0.38**	0.11
	(0.07)	(0.15)	(0.06)
Infomation Comprh.	−0.04	0.44	−0.04
	(1.15)	(0.30)	(0.13)
Learning Process	0.19	0.60**	−0.12
	(1.14)	(0.29)	(1.13)
R^2	0.262	0.283	0.165
Adj. R^2	0.184	0.206	0.075

Note: *p <0.1; **p <0.05; ***p <0.01.

while participants' information processing style, learning style, and familiarity with AI developer tools were modeled as independent variables to explain the variation in usage counts.

Results. Table 3 presents the results of the regression analysis conducted for three response variables. The first model (Prompt (1)), which uses the total count of prompt-based interactions (`prompt` + `prompt-context` + `prompt-followup`), reveals that developers who are more familiar with other AI developer tools are more likely to prompt the LLMs using natural language queries. This result confirms our hypothesis that the AI tool familiarity level influences developers' use of queries. The familiarity level also has a statistically significant impact on `prompt-followup`, as shown in the Followup model (2). However, we did not find any significant impact of participants' information processing style on their use of GILT. This means that selective processors and comprehensive processors probed the LLMs similarly, as far as we can tell. The model, however, shows a statistically significant correlation between participants' learning styles and `prompt-followup` feature usage. Specifically, process-oriented learners tend to probe LLMs more frequently than tinkerers. This result might indicate that process-oriented learners are more likely to learn thoroughly before proceeding to the next step, while tinkerers tend to tinker with the code after getting the minimum amount of direction from GILT. Finally, the All model (3), which uses the total count of all GILT interactions, indicates that there is no statistically significant difference between the information styles, learning styles, and familiarity levels in terms of overall feature usage counts.

Summary RQ2

Overall, participants used `Overview` and `Prompt-context` most frequently. However, the way participants interact with GILT varied based on their learning styles and familiarity with other AI tools.

7 RQ3: USER PERCEPTIONS

In this section, we investigate how participants perceived their experience of using GILT. Specifically, we examine their perceived usefulness, usability, and cognitive load in comparison to search-based information seeking. Additionally, we explore the pros and cons participants reported, and suggestions for improving the tool.

7.1 Comparison with Web Search

We employed two wildly-used standard measures, TLX and TAM, in our post-task survey and compared them using two-tailed paired t-tests. TAM (Technology Acceptance Model) [36] is a widely used survey that assesses users' acceptance and adoption of new technologies, and TLX (Task Load Index) [21] is a subjective measure of mental workload that considers several dimensions, including mental, physical, and temporal demand, effort, frustration, and performance. The summaries of TAM and TLX comparisons can be found in our replication package.

The average scores for the [perceived usefulness, perceived ease of use] in TAM scales were [27.3, 29.75] for the control condition, and [33.49, 34.2] for the treatment condition. The paired t-tests on the TAM scores indicated that there were significant differences in perceived usefulness and perceived usability scores between the two conditions ($p < 0.001$). Specifically, participants rated GILT higher on both dimensions than they did search engines.

For TLX items [mental demand, physical demand, temporal demand, performance, effort, frustration], the average scores were [3.8, -2.1, 4.0, 1.6, 3.4, -0.1] for the control condition and [3.3, -2.5, 2.6, 3.3, 3.3, 1.0] for the treatment condition. Paired t-tests on the TLX scores revealed statistically significant differences between the tool and search engines in temporal demand ($p < 0.05$) and performance ($p < 0.05$) but not in other items. These results indicate that the participants felt less rushed when using GILT than when using search engines, and they felt more successful in accomplishing the task with the tool than with search engines, but there were no significant differences in other dimensions.

7.2 User Feedback

In the post-task survey, we asked open-ended questions regarding (i) their general experience with using GILT, (ii) the tool's fit with the participants' workflow, (iii) comparison with other tools, and (iv) opportunities for improvement. Two authors conducted a thematic analysis [10] to analyze the answers. Initially, two authors separately performed open coding on the same set of 8 responses (25% of the entire data), and convened to discuss and merge the codes into a shared codebook. The first author coded the rest of the responses and discussed with the rest of the authors whenever new codes needed to be added. The codebook is available in our replication package, and we discuss some of them here.

The participants in this study reported several positive aspects of the tool, with the most notable being context incorporation. Participants valued the ability to prompt the LLM with their code as context, which allowed them to tailor the LLM's suggestions to their specific programming context, e.g., "the extension generated code that could easily be used in the context of the task I was performing, without much modification." (P5) Participants also found it extremely useful to prompt the LLM with just code context,

as it allowed them to bypass the need to write proficient queries, a well-known challenge in search-based information seeking [32, 74]. P15 mentioned "*It's nice not to need to know anything about the context before being effective in your search strategy.*"

Many participants reported that using the tool helped them speed up their information seeking, by reducing the need to forage for information, e.g., "Stack Overflow or a Google search would require more time and effort in order to find the exact issue and hence would be time-consuming." (P27)

Some participants, however, reported having a hard time finding a good prompt that could give them the desired response. Combined with the need for good prompts and the limitations of LLM, this led some participants to report that the responses provided by the tool were occasionally inaccurate, reducing their productivity. P28 summarized this issue well: "*[prototype] was not able to give me the code that I was looking for, so it took up all my time (which I got very annoyed about). I think I just didn't word the question well.*"

Participants had mixed opinions on the different features of the tool, especially the buttons. Some preferred to use "*different buttons for different types of information so I didn't have to read a lot of text to find what I was looking for*" (P7), while others thought that was overkill and mentioned "*a simpler view would be nice.*" (P8)

Compared to ChatGPT, 17 participants (out of 19 who answered) mentioned advantages of GILT, with the `Prompt- context` feature being one of the main ones. Participants expressed positive feelings about CoPilot but acknowledged that the tool had a different role than CoPilot and that they would be complementary to each other, e.g.: "*Copilot is a tool that I can complete mechanical works quickly, but [GILT] offers insight into more challenging tasks.*" (P29)

Many participants reported that the tool would be even more useful when combined with search engines, API documentation, or CoPilot,[2] as they provide different types of information than the tool. Having the ability to choose sources based on their needs would enhance their productivity by giving them control over the trade-offs, such as speed, correctness, and adaptability of the information.

Summary RQ3

Participants appreciated GILT's ability to easily incorporate their code as context, but some participants reported that writing a good prompt is still a challenge.

8 THREATS TO VALIDITY

One potential concern with our study design is the task and library selection. We only used tasks that show visible outputs, which might have led participants to detect potential errors more easily, compared to other tasks, such as optimization or parallel programming. However, we believe that the tasks we chose are representative of common programming errors that would need to be identified in real-world programming situations. Indeed, when we asked the participants in the post-task survey, both data visualization and 3D rendering tasks were reported to very or extremely closely resemble real-world tasks by 82% and 73% of the participants.

[2]Notably, GitHub independently announced these enhancements to Copilot already, after we conducted our study: https://www.theverge.com/2023/7/20/23801498/github-copilot-x-chat-code-chatbot-public-beta

Similarly, the selection of libraries might have biased the study results. However, in selecting libraries for our study, we avoided using popular libraries that could unintentionally give an advantage to LLMs. We believe that the libraries we chose are of medium size and quality, and therefore represent a fair test of the LLM tools. However, it is possible that different libraries or larger codebases could produce different results.

Despite our efforts to create a controlled experience, several factors differentiate our in-IDE extension from search engines, aside from the inclusion of LLMs. For example, although previous research investigating the incorporation of search into IDE did not find a statistically significant difference between the control and treatment groups [6, 39], the in-IDE design itself may have been more helpful than access to LLMs, as it potentially reduced context-switching. Thus, further studies are needed to gain a better understanding of the extent to which each benefit of our prototype can be attributed to these differences.

Additionally, the laboratory setting may not fully capture the complexity of real-world programming tasks, which could impact the generalizability of our findings. Also, the time pressure participants could have felt, and the novelty effect in a lab setting could have changed how users interact with LLMs. Our sample size, 32, was relatively small and skewed towards those in academia. This may also limit the generalizability of our findings to more professional programmers. Thus, future research with larger, more diverse samples is necessary to confirm and expand upon our results.

Our analysis also has the standard threats to statistical conclusion validity affecting regression models. Overall, we took several steps to increase the robustness of our estimated regression results. First, we removed outliers from the top 1% most extreme values. Second, we checked for multicollinearity using the Variation Influence Factor (VIF) and confirmed that all variables we used had VIF lower than 2.5 following Johnston et al. [28].

Another potential threat to the validity of our findings is the rapid pace of technological development in the field of LLM tools. Despite our efforts to use the most up-to-date LLM available at the time of the submission, it is possible that new breakthroughs could render our findings obsolete before long.

9 DISCUSSION AND IMPLICATIONS

Comprehension outsourcing. Our analysis revealed an intriguing finding regarding participants' behavior during the study, where some of them deferred their need for code comprehension to the LLM, which was well described by one participant as *comprehension outsourcing*. These participants prompted the model at a higher level directly and did not read and fully comprehend the code before making changes. As one participant commented, "*I was surprised by how little I had to know about (or even read) the starter code before I can jump in and make changes.*" This behavior might be attributed to developers' inclination to focus on task completion rather than comprehending the software, as reported in the literature [44]. Or, participants may have also weighed the costs and risks of comprehending code themselves, and chosen to defer their comprehension efforts to the language model. While this behavior was observed in the controlled setting of a lab study and may not fully reflect how developers approach code comprehension in their daily work, it

does raise concerns about the potential impact of such a trend (or over-reliance on LLMs [71]) on code quality. This highlights the importance of preventing developers who tend to defer their comprehension efforts to the LLM from being steered in directions that neither they nor the LLM are adequately equipped to handle. Studies showing developers' heavy reliance on Stack Overflow, despite its known limitations in accuracy and currency [73, 78], further emphasize the need for caution before widely adopting LLM-based tools in code development. Research on developers' motivations and reasons for code comprehension when LLMs are available will be valuable in informing future tool designs.

Need for more research in UI. In our analysis, we observed a notable trend where the professionals benefited *more* from the tool compared to students. Our examination of the prompts indicated that this discrepancy may arise because students face challenges in constructing effective queries or revising them to obtain useful information, aligning with findings in the literature on code generation using LLMs [13]. Although we provided an option to use prompt-less interaction with LLMs to reduce the difficulty in prompt engineering, a lot of participants chose to use prompt-based interaction, possibly due to their familiarity with other AI tools, the potentially higher quality of information this mode produces, or other reasons that our study did not cover. However, we find our results still promising, as we observed that students used prompt-less interaction more than the professionals and succeeded more when using the buttons than using the prompts. We believe that further research is needed, exploring various interaction options to support a diverse developer population.

Utilize more context. One of the main advantages of GILT reported by the participants is its ability to prompt the LLM with the code being edited as context. We believe that additional types of context can be leveraged to improve the tool's utility, including project context (e.g., project scale and domain), system context (e.g., programming languages and target deployment environments), and personal context (e.g., programming expertise in libraries, and domains). By combining these contexts with proper back-end engineering, we believe that GILT, or other LLM-powered developer tools, will be able to provide relevant information to developers with even less prompt engineering efforts of the users.

Need further studies in real-world settings. One possible explanation for some of the models with null results from RQ1 and RQ2 is the artificial setting of the lab study, where participants were encouraged to focus on small, specific task requirements instead of exploring the broader information dimension. For example, participants prioritized completing more tasks rather than fully understanding the code, as reported by participant P18 in their survey response: " *[GILT] ..., which could definitely help one to tackle the task better if there weren't under the timed-settings.*" Thus, although our first study shed some light on the potential challenges and promises, to fully understand the implications of deploying this tool into general developer pipelines, it is necessary to observe how programmers use it in real-world settings with larger-scale software systems, less specific goals, and over a longer time frame. Given that GitHub recently launched CopilotX [27], a tool that offers a comparable set of features to our prototype to enhance developer experience, such research is urgently needed. We believe that our findings are a timely contribution and a good first step for researchers and tool builders in designing and developing developer assistants that effectively use LLMs.

10 CONCLUSION

We presented the results of a user study that aimed to investigate the effectiveness of generation-based information support using LLMs to aid developers in code understanding. With our in-IDE prototype tool, GILT, we demonstrated that this approach significantly enhances developers' ability to complete tasks compared to traditional search-based information seeking. At the same time, we also identified that the degree of benefits developers can get from the tool differs between students and professionals, and the way developers interact with the tool varies based on their learning styles and familiarity with other AI tools.

Data Availability. Our supplementary material includes the replication package, including the study protocol, tasks, study data, scripts to replicate the analyses, as well as the prototype tool, GILT, are available online at DOI https://doi.org/10.5281/zenodo.10461385 [50].

11 ACKNOWLEDGEMENT

We would like to thank our participants for their time and input, and our reviewers for their valuable feedback.

REFERENCES

[1] Matin Amoozadeh, David Daniels, Daye Nam, Stella Chen, Michael Hilton, Sruti Srinivasa Ragavan, and Mohammad Amin Alipour. 2023. Trust in Generative AI among students: An Exploratory Study. arXiv:2310.04631 Retrieved from https://arxiv.org/abs/2310.04631.

[2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1 (2023), 85–111. https://doi.org/10.1145/3586030

[3] Celeste Barnaby, Koushik Sen, Tianyi Zhang, Elena Glassman, and Satish Chandra. 2020. Exempla Gratis (E.G.): Code Examples for Free. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020), Virtual Event, USA.* ACM, New York, NY, 1353–1364. https://doi.org/10.1145/3368089.3417052

[4] Laura Beckwith, Cory Kissinger, Margaret M. Burnett, Susan Wiedenbeck, Joseph Lawrance, Alan F. Blackwell, and Curtis R. Cook. 2006. Tinkering and gender in end-user programmers' debugging. In *Proceedings of the 2006 Conference on Human Factors in Computing Systems, (CHI 2006), Montréal, Québec, Canada, April 22-27, 2006.* ACM, New York, NY, 231–240. https://doi.org/10.1145/1124772.1124808

[5] Bokeh. 2023. Bokeh. https://bokeh.org/ Retrieved: 2024-01-11.

[6] Joel Brandt, Mira Dontcheva, Marcos Weskamp, and Scott R. Klemmer. 2010. Example-centric programming: integrating web search into the development environment. In *Proceedings of the 28th International Conference on Human Factors in Computing Systems, (CHI 2010), Atlanta, Georgia, USA, April 10-15, 2010.* ACM, New York, NY, 513–522. https://doi.org/10.1145/1753326.1753402

[7] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two studies of opportunistic programming: interleaving web foraging, learning, and writing code. In *Proceedings of the 27th International Conference on Human Factors in Computing Systems, (CHI 2009), Boston, MA, USA, April 4-9, 2009.* ACM, New York, NY, 1589–1598. https://doi.org/10.1145/1518701.1518944

[8] Margaret M. Burnett, Simone Stumpf, Jamie Macbeth, Stephann Makri, Laura Beckwith, Irwin Kwan, Anicia Peters, and Will Jernigan. 2016. GenderMag: A Method for Evaluating Software's Gender Inclusiveness. *Interact. Comput.* 28, 6 (2016), 760–787. https://doi.org/10.1093/IWC/IWV046

[9] Jie-Cherng Chen and Sun-Jen Huang. 2009. An empirical analysis of the impact of software development problem factors on software maintainability. *J. Syst. Softw.* 82, 6 (2009), 981–992. https://doi.org/10.1016/J.JSS.2008.12.036

[10] Victoria Clarke and Virginia Braun. 2013. Teaching thematic analysis: Overcoming challenges and developing strategies for effective learning. *The psychologist* 26, 2 (2013), 120–123.

[11] William K Darley and Robert E Smith. 1995. Gender differences in information processing strategies: An empirical test of the selectivity model in advertising response. *Journal of advertising* 24, 1 (1995), 41–56.

[12] Matthew C. Davis, Emad Aghayi, Thomas D. LaToza, Xiaoyin Wang, Brad A. Myers, and Joshua Sunshine. 2023. What's (Not) Working in Programmer User Studies? *ACM Trans. Softw. Eng. Methodol.* 32, 5 (2023), 120:1–120:32. https://doi.org/10.1145/3587157

[13] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education, Volume 1, (SIGCSE 2023), Toronto, ON, Canada, March 15-18, 2023*. ACM, New York, NY, USA, 1136–1142. https://doi.org/10.1145/3545945.3569823

[14] Ekwa Duala-Ekoko and Martin P Robillard. 2010. *The information gathering strategies of API learners*. Technical Report. TR-2010.6, School of Computer Science, McGill University.

[15] Ekwa Duala-Ekoko and Martin P. Robillard. 2012. Asking and answering questions about unfamiliar APIs: An exploratory study. In *34th International Conference on Software Engineering, (ICSE 2012), June 2-9, 2012, Zurich, Switzerland*. IEEE Computer Society, 266–276. https://doi.org/10.1109/ICSE.2012.6227187

[16] Jean-Baptiste Döderlein, Mathieu Acher, Djamel Eddine Khelladi, and Benoit Combemale. 2023. Piloting Copilot and Codex: Hot Temperature, Cold Prompts, or Black Magic? arXiv:2210.14699 Retrieved from https://arxiv.org/abs/2210.14699.

[17] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Australasian Computing Education Conference (ACE 2022) Virtual Event, Australia, February 14 - 18, 2022*. ACM, 10–19. https://doi.org/10.1145/3511861.3511863

[18] Luanne Freund. 2015. Contextualizing the information-seeking behavior of software engineers. *J. Assoc. Inf. Sci. Technol.* 66, 8 (2015), 1594–1605. https://doi.org/10.1002/ASI.23278

[19] Valentina Grigoreanu, Margaret M. Burnett, and George G. Robertson. 2010. A strategy-centric approach to the design of end-user debugging tools. In *Proceedings of the 28th International Conference on Human Factors in Computing Systems, (CHI 2010), Atlanta, Georgia, USA, April 10-15, 2010*. ACM, 713–722. https://doi.org/10.1145/1753326.1753431

[20] Md Montaser Hamid, Amreeta Chatterjee, Mariam Guizani, Andrew Anderson, Fatima Moussaoui, Sarah Yang, I Escobar, Anita Sarma, and Margaret Burnett. 2023. How to measure diversity actionably in technology. *Equity, Diversity, and Inclusion in Software Engineering: Best Practices and Insights* (2023).

[21] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.

[22] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutcheme, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the Responses of Large Language Models to Beginner Programmers' Help Requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER 2023)*. ACM. https://doi.org/10.1145/3568813.3600139

[23] Amber Horvath, Sachin Grover, Sihan Dong, Emily Zhou, Finn Voichick, Mary Beth Kery, Shwetha Shinju, Daye Nam, Mariann Nagy, and Brad A. Myers. 2019. The Long Tail: Understanding the Discoverability of API Functionality. In *2019 IEEE Symposium on Visual Languages and Human-Centric Computing, (VL/HCC 2019), Memphis, Tennessee, USA, October 14-18, 2019*. IEEE Computer Society, 157–161. https://doi.org/10.1109/VLHCC.2019.8818681

[24] Saki Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-programming? An Empirical Study. In *44th IEEE/ACM International Conference on Software Engineering: Companion Proceedings, (ICSE Companion 2022), Pittsburgh, PA, USA, May 22-24, 2022*. ACM/IEEE, 319–321. https://doi.org/10.1145/3510454.3522684

[25] GitHub Inc. 2024. GitHub Codespaces. https://github.com/features/codespaces Retrieved: 2024-01-11.

[26] GitHub Inc. 2024. GitHub Copilot. https://github.com/features/copilot Retrieved: 2024-01-11.

[27] GitHub Inc. 2024. GitHub Copilot X: The AI-powered developer experience. https://github.com/features/preview/copilot-x Retrieved: 2024-01-11.

[28] Ron Johnston, Kelvyn Jones, and David Manley. 2018. Confounding and collinearity in regression analysis: a cautionary tale and an alternative procedure, illustrated by studies of British voting behaviour. *Quality & quantity* 52 (2018), 1957–1976.

[29] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, (CHI 2023), Hamburg, Germany, April 23-28, 2023*. ACM, 455:1–455:23. https://doi.org/10.1145/3544548.3580919

[30] Amy J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *29th International Conference on Software Engineering (ICSE 2007), Minneapolis, MN, USA, May 20-26, 2007*. IEEE Computer Society, 344–353. https://doi.org/10.1109/ICSE.2007.45

[31] Amy J. Ko, Brad A Myers, Michael J Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (11 2006), 971 – 987. https://doi.org/10.1109/tse.2006.116

[32] Amy J. Ko and Yann Riche. 2011. The role of conceptual knowledge in API usability. In *2011 IEEE Symposium on Visual Languages and Human-Centric Computing, (VL/HCC 2011), Pittsburgh, PA, USA, September 18-22, 2011*. IEEE, 173–176. https://doi.org/10.1109/VLHCC.2011.6070395

[33] Thomas D. LaToza, David Garlan, James D. Herbsleb, and Brad A. Myers. 2007. Program comprehension as fact finding. In *Proceedings of the 6th joint meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE 2007), Dubrovnik, Croatia, September 3-7, 2007*. ACM, 361–370. https://doi.org/10.1145/1287624.1287675

[34] Thomas D LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: a study of developer work habits. In *Proceedings of the 28th international conference on Software engineering*. 492–501.

[35] Joseph Lawrance, Christopher Bogart, Margaret M. Burnett, Rachel K. E. Bellamy, Kyle Rector, and Scott D. Fleming. 2013. How Programmers Debug, Revisited: An Information Foraging Theory Perspective. *IEEE Trans. Software Eng.* 39, 2 (2013), 197–215. https://doi.org/10.1109/TSE.2010.111

[36] Younghwa Lee, Kenneth A Kozar, and Kai RT Larsen. 2003. The technology acceptance model: Past, present, and future. *Communications of the Association for information systems* 12, 1 (2003), 50.

[37] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing Code Explanations Created by Students and Large Language Models. *arXiv* (2023). arXiv:2304.03938

[38] Hongwei Li, Zhenchang Xing, Xin Peng, and Wenyun Zhao. 2013. What help do developers seek, when and how?. In *20th Working Conference on Reverse Engineering, (WCRE 2013), Koblenz, Germany, October 14-17, 2013*. IEEE Computer Society, 142–151. https://doi.org/10.1109/WCRE.2013.6671289

[39] Hongwei Li, Xuejiao Zhao, Zhenchang Xing, Lingfeng Bao, Xin Peng, Dongjing Gao, and Wenyun Zhao. 2015. amAssist: In-IDE ambient search of online programming resources. In *22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering, (SANER 2015), Montreal, QC, Canada, March 2-6, 2015*. IEEE Computer Society, 390–398. https://doi.org/10.1109/SANER.2015.7081849

[40] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2023. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. arXiv:2303.17125 Retrieved from https://arxiv.org/abs/2303.17125.

[41] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210 Retrieved from https://arxiv.org/abs/2305.01210.

[42] Michael Xieyang Liu, Aniket Kittur, and Brad A. Myers. 2022. Crystalline: Lowering the Cost for Developers to Collect and Organize Information for Decision Making. In *Conference on Human Factors in Computing Systems (CHI 2022), New Orleans, LA, USA, 29 April 2022 - 5 May 2022*. ACM, 68:1–68:16. https://doi.org/10.1145/3491102.3501968

[43] Walid Maalej and Martin P. Robillard. 2013. Patterns of Knowledge in API Reference Documentation. *IEEE Trans. Software Eng.* 39, 9 (2013), 1264–1282. https://doi.org/10.1109/TSE.2013.12

[44] Walid Maalej, Rebecca Tiarks, Tobias Roehm, and Rainer Koschke. 2014. On the Comprehension of Program Comprehension. *ACM Trans. Softw. Eng. Methodol.* 23, 4 (2014), 31:1–31:37. https://doi.org/10.1145/2622669

[45] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2022. Experiences from Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book. *arXiv* (2022). arXiv:2211.02265

[46] Michael Meng, Stephanie Steinhardt, and Andreas Schubert. 2017. Application Programming Interface Documentation: What Do Software Developers Want?:. *Journal of Technical Writing and Communication* 48, 3 (07 2017), 295 – 330. https://doi.org/10.1177/0047281617721853

[47] Michael Meng, Stephanie Steinhardt, and Andreas Schubert. 2019. How Developers Use API Documentation: An Observation Study. *Commun. Des. Q. Rev* 7, 2 (aug 2019), 40–49. https://doi.org/10.1145/3358931.3358937

[48] Andre N. Meyer, Laura E. Barton, Gail C. Murphy, Thomas Zimmermann, and Thomas Fritz. 2017. The Work Life of Developers: Activities, Switches and Perceived Productivity. *IEEE Transactions on Software Engineering* 43, 12 (2017), 1178–1193. https://doi.org/10.1109/tse.2017.2656886

[49] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2023. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. arXiv:2210.14306 Retrieved from https://arxiv.org/abs/2210.14306.

[50] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Artifacts for Using an LLM to Help With Code Understanding. https://doi.org/10.5281/zenodo.10461385

[51] Daye Nam, Andrew Macvean, Brad Myers, and Bogdan Vasilescu. 2023. Exploring Documentation Usage via Page-view Log Analysis. arXiv:2310.10817 Retrieved from https://arxiv.org/abs/2310.10817.

[52] Daye Nam, Brad A. Myers, Bogdan Vasilescu, and Vincent J. Hellendoorn. 2023. Improving API Knowledge Discovery with ML: A Case Study of Comparable API Methods. In *45th IEEE/ACM International Conference on Software Engineering, (ICSE 2023), Melbourne, Australia, May 14-20, 2023*. IEEE, 1890–1906. https:

//doi.org/10.1109/ICSE48619.2023.00161
[53] Open3D. 2024. Open3D – A Modern Library for 3D Data Processing. http://www.open3d.org/ Retrieved: 2024-01-11.
[54] OpenAI. 2024. ChatGPT|OpenAI. https://chat.openai.com/ Retrieved: 2024-01-11.
[55] David N Perkins, Chris Hancock, Renee Hobbs, Fay Martin, and Rebecca Simmons. 1986. Conditions of learning in novice programmers. *Journal of Educational Computing Research* 2, 1 (1986), 37–55.
[56] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2022. Do Users Write More Insecure Code with AI Assistants? *CoRR* abs/2211.03622 (2022). https://doi.org/10.48550/arXiv.2211.03622 arXiv:2211.03622
[57] David Piorkowski, Austin Z. Henley, Tahmid Nabi, Scott D. Fleming, Christopher Scaffidi, and Margaret M. Burnett. 2016. Foraging and navigations, fundamentally: developers' predictions of value and cost. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE 2016), Seattle, WA, USA, November 13-18, 2016*. ACM, 97–108. https://doi.org/10.1145/2950290.2950302
[58] Md. Masudur Rahman, Jed Barson, Sydney Paul, Joshua Kayani, Federico Andres Lois, Sebastian Fernandez Quezada, Christopher Parnin, Kathryn T. Stolee, and Baishakhi Ray. 2018. Evaluating how developers use general-purpose web-search for code retrieval. In *Proceedings of the 15th International Conference on Mining Software Repositories, (MSR 2018), Gothenburg, Sweden, May 28-29, 2018*. ACM, 465–475. https://doi.org/10.1145/3196398.3196425
[59] Nikitha Rao, Chetan Bansal, Thomas Zimmermann, Ahmed Hassan Awadallah, and Nachiappan Nagappan. 2020. Analyzing Web Search Behavior for Software Engineering Tasks. In *2020 IEEE International Conference on Big Data (IEEE BigData 2020), Atlanta, GA, USA, December 10-13, 2020*. IEEE, 768–777. https://doi.org/10.1109/BIGDATA50022.2020.9378083
[60] Martin P. Robillard. 2009. What Makes APIs Hard to Learn? Answers from Developers. *IEEE Softw.* 26, 6 (2009), 27–34. https://doi.org/10.1109/MS.2009.193
[61] Martin P. Robillard and Robert DeLine. 2011. A field study of API learning obstacles. *Empir. Softw. Eng.* 16, 6 (2011), 703–732. https://doi.org/10.1007/S10664-010-9150-8
[62] Tobias Roehm. 2015. Two user perspectives in program comprehension: end users and developer users. In *Proceedings of the 2015 IEEE 23rd International Conference on Program Comprehension, (ICPC 2015), Florence/Firenze, Italy, May 16-24, 2015*. IEEE Computer Society, 129–139. https://doi.org/10.1109/ICPC.2015.22
[63] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael J. Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces, (IUI 2023), Sydney, NSW, Australia, March 27-31, 2023*. ACM, 491–514. https://doi.org/10.1145/3581641.3584037
[64] Advait Sarkar, Carina Negreanu, Ben Zorn, Sruti Srinivasa Ragavan, Christian Pölitz, and Andrew D. Gordon. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group, (PPIG 2022), The Open University, Milton Keynes, UK & Online, September 5-9, 2022*. Psychology of Programming Interest Group, 127–153. https://ppig.org/papers/2022-ppig-33rd-sarkar/
[65] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *ACM Conference on International Computing Education Research (ICER 2022), Lugano and Virtual Event, Switzerland, August 7 - 11, 2022, Volume 1*. ACM, 27–43. https://doi.org/10.1145/3501385.3543957
[66] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. 2008. Asking and Answering Questions during a Programming Change Task. *IEEE Trans. Software Eng.* 34, 4 (2008), 434–451. https://doi.org/10.1109/TSE.2008.26
[67] Tabnine. 2024. Tabnine: AI assistant for software developers. https://www.tabnine.com/ Retrieved: 2024-01-11.
[68] Kyle Thayer, Sarah E Chasins, and Amy J Ko. 2021. A Theory of Robust API Knowledge. *ACM Transactions on Computing Education* 21, 1 (2021), 1–32. https://doi.org/10.1145/3444945
[69] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F. Bissyandé. 2023. Is ChatGPT the Ultimate Programming Assistant – How far is it? arXiv:2304.11938 Retrieved from https://arxiv.org/abs/2304.11938.
[70] Gias Uddin and Martin P. Robillard. 2015. How API Documentation Fails. *IEEE Softw.* 32, 4 (2015), 68–75. https://doi.org/10.1109/MS.2014.80
[71] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems (CHI 2022), New Orleans, LA, USA, 29 April 2022 - 5 May 2022, Extended Abstracts*. ACM, 332:1–332:7. https://doi.org/10.1145/3491101.3519665
[72] Shaowei Wang, Tse-Hsun Chen, and Ahmed E. Hassan. 2020. How Do Users Revise Answers on Technical Q&A Websites? A Case Study on Stack Overflow. *IEEE Trans. Software Eng.* 46, 9 (2020), 1024–1038. https://doi.org/10.1109/TSE.2018.2874470
[73] Yuhao Wu, Shaowei Wang, Cor-Paul Bezemer, and Katsuro Inoue. 2019. How do developers utilize source code from stack overflow? *Empir. Softw. Eng.* 24, 2 (2019), 637–673. https://doi.org/10.1007/S10664-018-9634-5
[74] Xin Xia, Lingfeng Bao, David Lo, Pavneet Singh Kochhar, Ahmed E. Hassan, and Zhenchang Xing. 2017. What do developers search for on the web? *Empir. Softw. Eng.* 22, 6 (2017), 3149–3185. https://doi.org/10.1007/S10664-017-9514-4
[75] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *6th ACM SIGPLAN International Symposium on Machine Programming (MAPS@PLDI 2022), San Diego, CA, USA, 13 June 2022*. ACM, 1–10. https://doi.org/10.1145/3520312.3534862
[76] J. D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI 2023), Hamburg, Germany, April 23-28, 2023*. ACM, 437:1–437:21. https://doi.org/10.1145/3544548.3581388
[77] Tianyi Zhang, Björn Hartmann, Miryung Kim, and Elena L. Glassman. 2020. Enabling Data-Driven API Design with Community Usage Data: A Need-Finding Study. In *CHI Conference on Human Factors in Computing Systems (CHI 2020), Honolulu, HI, USA, April 25-30, 2020*. ACM, 1–13. https://doi.org/10.1145/3313831.3376382
[78] Tianyi Zhang, Ganesha Upadhyaya, Anastasia Reinhardt, Hridesh Rajan, and Miryung Kim. 2018. Are code examples on an online Q&A forum reliable?: a study of API misuse on stack overflow. In *Proceedings of the 40th International Conference on Software Engineering (ICSE 2018), Gothenburg, Sweden, May 27 - June 03, 2018*. ACM, 886–896. https://doi.org/10.1145/3180155.3180260
[79] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *6th ACM SIGPLAN International Symposium on Machine Programming (MAPS@PLDI 2022), San Diego, CA, USA, 13 June 2022*. ACM, 21–29. https://doi.org/10.1145/3520312.3534864